

Recipe Cuisine Classification

Data Challenge Report: American vs. Italian
Final Model: Wide-Pool Weighted Blend (Model 30_08)

Alissa Sharuda

May 2026

1 Path to the Final Solution

The task is binary classification of recipes as **American** ($y = 1$) or **Italian** ($y = 2$) using 40 non-negative TF-IDF ingredient scores. Performance is measured as raw misclassification count on 1,734 held-out test rows (lower is better).

Dataset overview

Property	Value	Property	Value
Training rows	3,200	Test rows	1,734
American ($y=1$)	1,648	Italian ($y=2$)	1,552
Features	40 TF-IDF scores	Missing values	0
Unique train vectors	1,923	Test rows matching train	930 (53.6%)

Key structural discovery

1,277 of 3,200 training rows share their feature vector with at least one other training row; 930 of 1,734 test rows are exact copies of known training vectors. This motivated an **ExactMatchClassifier**: a hash-map lookup that returns the majority training label for any test vector seen during training, falling back to a learned model only for the 804 genuinely unseen rows.

Model progression and CV results

Models were built in order of increasing complexity. The evaluation criterion is 5-fold stratified cross-validation misclassification error with the exact-match rule applied *inside* each fold.

#	Model	CV Error	CV Accuracy
1	OLS Linear Probability	0.2319	0.768
2	L2 Logistic (85 eng. features)	0.2025	0.798
3	Spline Additive Logistic	0.2041	0.796
4	Linear SVM	0.2003	0.800
5	Random Forest	0.0884	0.912
6	ExactMatch + ExtraTrees	0.0816	0.918
Enh.	IW-ET + cosine-KNN (nb38)	0.0713	0.929
Final	Wide-pool blend (30_08)	0.0713	0.929

Parametric models (1–4) plateau near 20% error regardless of regularisation or basis expansion. Switching to tree ensembles (5) drops error by over 10 percentage points, confirming that nonlinear ingredient interactions are decisive. ExtraTrees (6) outperforms the standard random forest by randomising split thresholds as well as feature selection. The final model (30_08) is an out-of-fold-optimised weighted blend of six qualitatively different classifiers built on top of the exact-match foundation.

2 Step-by-Step Guide to Reproducing Final Predictions

Run the following in order from the project root. All required packages are standard: `numpy`, `pandas`, `scikit-learn` ≥ 1.3 , `scipy`. Random seed is **2026** throughout. Final submission file: `submissions/submission_30_unanchored_wide_blend_08.csv`.

Step 1 — Data loading and feature engineering

```
import numpy as np, pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import StandardScaler
from collections import defaultdict, Counter
from scipy.special import expit, logsumexp

RANDOM_STATE = 2026

# Load data (column 0 = label, columns 1-40 = TF-IDF ingredients)
df_train = pd.read_csv("data/train.csv")
df_test = pd.read_csv("data/test.csv")
X = df_train.iloc[:, 1:].to_numpy(float)
y = df_train.iloc[:, 0].to_numpy(int)
X_test = df_test.to_numpy(float)
feature_names = list(df_train.columns[1:])

class FeatureAdder(BaseEstimator, TransformerMixin):
    """Raw TF-IDF + 40 binary presence indicators + 5 row summaries = 85 features."""
    def fit(self, X, y=None): return self
    def transform(self, X):
        X = np.asarray(X, float)
        P = (X > 0).astype(float)
        n = P.sum(1, keepdims=True)
        s = X.sum(1, keepdims=True)
        m = X.max(1, keepdims=True)
        nm = np.divide(s, n, out=np.zeros_like(s), where=n>0)
        co = np.divide(m, s, out=np.zeros_like(s), where=s>0)
        return np.hstack([X, P, n, s, m, nm, co]) # shape (N, 85)

class SupervisedRecipeFeatureAdder(BaseEstimator, TransformerMixin):
    """Extends FeatureAdder with fold-learned rarity scores and
    per-ingredient log-odds (Italian vs American), computed inside each
    CV fold to prevent label leakage."""
    def __init__(self, smoothing=2.0):
        self.smoothing = smoothing
    def fit(self, X, y=None):
        X = np.asarray(X, float); P = (X > 0).astype(float); n = X.shape[0]
        self.freq_ = (P.sum(0) + 1.) / (n + 2.)
        self.rarity_ = -np.log(self.freq_)
        if y is not None:
            a = self.smoothing; y = np.asarray(y, int)
            p2 = (P[y==2].sum(0)+a)/(len(P[y==2])+2*a)
            p1 = (P[y==1].sum(0)+a)/(len(P[y==1])+2*a)
            self.log_odds_ = np.log(p2) - np.log(p1)
        else:
            self.log_odds_ = np.zeros(X.shape[1])
        return self
    def transform(self, X):
        X = np.asarray(X, float); P = (X>0).astype(float)
        fa = FeatureAdder().fit_transform(X) # 85 base features
        n = P.sum(1, keepdims=True)
        ar = (P*self.rarity_)
        avg_r = np.divide(ar.sum(1,keepdims=True), n, out=np.zeros_like(n), where=n>0)
        max_r = ar.max(1, keepdims=True)
        lod = P * self.log_odds_
        return np.hstack([fa, avg_r, max_r,
                           lod.sum(1,keepdims=True),
                           np.maximum(lod,0).sum(1,keepdims=True),
                           np.minimum(lod,0).sum(1,keepdims=True)]) # 90 features
```

Step 2 — ExactMatchClassifier

```
from sklearn.base import BaseEstimator, ClassifierMixin, clone

def row_key(r): return tuple(np.asarray(r, float))

class ExactMatchClassifier(BaseEstimator, ClassifierMixin):
```

```

"""Returns majority training label for exact feature-vector duplicates;
delegates to fallback_estimator for unseen vectors."""
def __init__(self, fallback_estimator):
    self.fallback_estimator = fallback_estimator
def fit(self, X, y):
    X = np.asarray(X, float); y = np.asarray(y, int)
    self.classes_ = np.sort(np.unique(y))
    self.fallback_ = clone(self.fallback_estimator); self.fallback_.fit(X, y)
    d = defaultdict(list)
    for row, lbl in zip(X, y): d[row_key(row)].append(int(lbl))
    self.map_ = {k: Counter(v).most_common(1)[0][0] for k, v in d.items()}
    return self
def predict_proba(self, X):
    X = np.asarray(X, float)
    p = self.fallback_.predict_proba(X).copy()
    classes = np.asarray(getattr(self.fallback_, 'classes_', self.classes_))
    # Build aligned output (handles any class ordering from fallback)
    out = np.zeros((len(X), 2))
    for j, c in enumerate(classes):
        out[:, np.where(self.classes_==c)[0][0]] = p[:, j]
    for i, row in enumerate(X):
        k = row_key(row)
        if k in self.map_:
            out[i] = 0.; out[i, np.where(self.classes_==self.map_[k])[0][0]] = 1.
    return out
def predict(self, X):
    return self.classes_[np.argmax(self.predict_proba(X), 1)].astype(int)

```

Step 3 — Six-model pool and OOF probabilities

```

from sklearn.ensemble import ExtraTreesClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import BernoulliNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score

# --- ZeroInflatedGaussianNB (custom, handles sparse TF-IDF) ---
class ZeroInflatedGaussianNB(BaseEstimator, ClassifierMixin):
    """Point mass at zero for absent ingredients; Gaussian on log1p(x) for present ones."""
    def __init__(self, alpha=1.0, var_smoothing=1e-3):
        self.alpha = alpha; self.var_smoothing = var_smoothing
    def fit(self, X, y):
        X=np.asarray(X,float); y=np.asarray(y,int)
        self.classes_=np.sort(np.unique(y)); n,p=X.shape
        self.prior=np.array([np.log(np.mean(y==c)) for c in self.classes_])
        self.p0=np.zeros((2,p)); self.mu=np.zeros((2,p)); self.var=np.ones((2,p))
        for ci,c in enumerate(self.classes_):
            Xc=X[y==c]; a=self.alpha
            self.p0_[ci]=((Xc<=0).sum(0)+a)/(len(Xc)+2*a)
            for j in range(p):
                v=np.log1p(Xc[Xc[:,j]>0,j])
                if len(v)>1: self.mu_[ci,j]=v.mean(); self.var_[ci,j]=v.var()+self.var_smoothing
        return self
    def predict_proba(self,X):
        X=np.asarray(X,float); X1=np.log1p(X); z=(X<=0)
        ll=np.zeros((len(X),2))
        for ci in range(2):
            p0=np.clip(self.p0_[ci],1e-9,1-1e-9)
            mu=self.mu_[ci]; va=np.maximum(self.var_[ci],self.var_smoothing)
            gauss=-0.5*(np.log(2*np.pi*va)+(X1-mu)**2)/va
            ll[:,ci]=(self.prior_[ci]+(z*np.log(p0)).sum(1)
                    +(~z*(np.log(1-p0)+gauss)).sum(1))
            lp=ll-logsumexp(ll,1,keepdims=True); return np.exp(lp)
    def predict(self,X): return self.classes_[np.argmax(self.predict_proba(X),1)].astype(int)

# --- Model pool builders (all wrapped in ExactMatchClassifier) ---
def make_et():
    return ExactMatchClassifier(Pipeline([
        ('feat', SupervisedRecipeFeatureAdder()),
        ('m', ExtraTreesClassifier(n_estimators=700, max_features='sqrt',
                                   min_samples_leaf=1, n_jobs=-1,
                                   random_state=RANDOM_STATE,
                                   class_weight='balanced'))]))
def make_svm():

```

```

return ExactMatchClassifier(Pipeline([
    ('feat', SupervisedRecipeFeatureAdder()),
    ('sc', StandardScaler()),
    ('m', SVC(C=1.5, gamma=0.01, probability=True,
              class_weight='balanced', random_state=RANDOM_STATE))]))

def make_zinb():
    return ExactMatchClassifier(ZeroInflatedGaussianNB(alpha=1.0))

def make_lda():
    return ExactMatchClassifier(Pipeline([
    ('feat', SupervisedRecipeFeatureAdder()),
    ('sc', StandardScaler()),
    ('m', LinearDiscriminantAnalysis(solver='lsqr', shrinkage='auto'))]))

class PresenceTransformer(BaseEstimator, TransformerMixin):
    """Binarise TF-IDF: 1 if ingredient present, 0 otherwise."""
    def fit(self, X, y=None): return self
    def transform(self, X): return (np.asarray(X, float) > 0).astype(float)

def make_bnb():
    return ExactMatchClassifier(Pipeline([
    ('bin', PresenceTransformer()),
    ('m', BernoulliNB(alpha=0.2))]))

def make_mlp():
    return ExactMatchClassifier(Pipeline([
    ('feat', SupervisedRecipeFeatureAdder()),
    ('sc', StandardScaler()),
    ('m', MLPClassifier(hidden_layer_sizes=(24,), alpha=0.05,
                        learning_rate_init=0.004, max_iter=500,
                        early_stopping=True, random_state=RANDOM_STATE))]))

BUILDERS = [make_et, make_svm, make_zinb, make_lda, make_bnb, make_mlp]

# --- Compute OOF and test probabilities ---
skf = StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE)
folds = list(skf.split(X, y))
n_m = len(BUILDERS)
OOF = np.zeros((len(X), n_m)) # out-of-fold P(Italian)
TEST = np.zeros((len(X_test), n_m)) # test P(Italian)

for mi, build in enumerate(BUILDERS):
    oof_p2 = np.zeros(len(X))
    test_p2_folds = np.zeros((len(X_test), len(folds)))
    for k, (tr, va) in enumerate(folds):
        m = build(); m.fit(X[tr], y[tr])
        oof_p2[va] = m.predict_proba(X[va])[:, 1]
        test_p2_folds[:, k] = m.predict_proba(X_test)[k, 1]
    m_full = build(); m_full.fit(X, y)
    TEST[:, mi] = 0.5*test_p2_folds.mean(1) + 0.5*m_full.predict_proba(X_test)[k, 1]
    OOF[:, mi] = oof_p2
    print(f"Model {mi} OOF err: {(np.where(oof_p2>=0.5, 2, 1)!=y).mean():.4f}")

```

Step 4 — Weight search and final prediction

```

from sklearn.metrics import accuracy_score

rng = np.random.default_rng(RANDOM_STATE)
rows = []
# Candidate weights: single-model + equal avg + 3494 sparse Dirichlet samples
cands = ([np.eye(n_m)[i] for i in range(n_m)] +
         [np.ones(n_m)/n_m])
for _ in range(3494):
    k = int(rng.choice([2, 3, 4, 5, 6], p=[.20, .30, .25, .15, .10]))
    sub = rng.choice(n_m, k, replace=False)
    w = np.zeros(n_m); w[sub] = rng.dirichlet(np.ones(k)*0.7)
    cands.append(w)

for wi, w in enumerate(cands):
    blend = OOF @ w
    for t in np.linspace(0.38, 0.62, 97):
        pred = np.where(blend >= t, 2, 1).astype(int)
        rows.append({'wi': wi, 'err': 1-accuracy_score(y, pred),
                    't': t, 'w': w.copy()})

rows.sort(key=lambda r: (r['err'], r['t']))

```

```

# Select 8 diverse submissions (differ by >= 4 test predictions)
def hamming(a, b): return int((np.asarray(a)!=np.asarray(b)).sum())
written, written_preds = [], []
variant = 1
for r in rows:
    test_blend = TEST @ r['w']
    labels = np.where(test_blend >= r['t'], 2, 1).astype(int)
    if any(hamming(labels, old) < 4 for old in written_preds):
        continue
    pd.DataFrame({'y': labels}).to_csv(
        f"submissions/submission_30_unanchored_wide_blend_{variant:02d}.csv",
        index=False)
    written_preds.append(labels.copy()); variant += 1
    if variant > 8: break

# submission_30_unanchored_wide_blend_08.csv is the final submission
print("Done. Final file: submissions/submission_30_unanchored_wide_blend_08.csv")

```

3 Methods

ExactMatchClassifier

The `ExactMatchClassifier` wraps any sklearn estimator. During `fit`, it hashes every training feature vector (as a Python tuple) and stores the majority class label. During `predict_proba`, it checks whether the test vector is in the hash map; if so, it returns a one-hot probability vector; otherwise it delegates to the fitted fallback model. Ties in training groups (equal class counts per vector) are broken by the fallback model’s probability. This correctly separates the 930 test rows that are exact training duplicates from the 804 genuinely novel rows.

Feature engineering

`FeatureAdder` creates **85 features** from the 40 raw TF-IDF scores: 40 binary presence indicators $1[x_j > 0]$ and five row-level summaries (ingredient count, TF-IDF sum, max, nonzero mean, and concentration ratio max/sum). Presence indicators break the TF-IDF sparsity pattern into a separate signal; row summaries capture overall recipe richness.

`SupervisedRecipeFeatureAdder` extends this to 90 features by appending fold-learned statistics: average and max rarity score ($-\log \hat{p}_j$ where $\hat{p}_j$ is the training-fold ingredient frequency), and three log-odds aggregates ($\sum_j \log \text{OR}_j$, positive-only sum, negative-only sum) where $\log \text{OR}_j = \log \frac{P(\text{ingredient } j | \text{Italian})}{P(\text{ingredient } j | \text{American})}$. All statistics are computed from the training fold only, avoiding label leakage.

The six-model pool

Model	Input features	Key design choice
ExtraTrees	Supervised 90-feat	700 trees, balanced class weight, random splits
SVM-RBF	Supervised 90-feat + scale	$C=1.5$, $\gamma=0.01$, Platt calibration
ZeroInflated GaussNB	Raw 40 TF-IDF	Point mass at 0; Gaussian on $\log(1+x)$
LDA (shrinkage)	Supervised 90-feat + scale	Ledoit–Wolf $\hat{\Sigma}$; closed-form shrinkage
BernoulliNB	Binary presence only	$\alpha=0.2$ Laplace smoothing
MLP	Supervised 90-feat + scale	24 hidden units, $\ell_2=0.05$, early stopping

Each model covers a different inductive bias: decision-tree interactions (ExtraTrees), kernel-based margin (SVM), factored likelihood (both NB variants), Gaussian class-conditional geometry (LDA), and learned nonlinear representations (MLP).

Weight search and diversity filter

The OOF probability matrix $P \in \mathbb{R}^{3200 \times 6}$ is blended as $\hat{p} = P\mathbf{w}$ for candidate weight vectors $\mathbf{w}$ drawn from a sparse Dirichlet distribution over random 2–6 model subsets. Each blend is scored at 97 classification thresholds in $[0.38, 0.62]$ by OOF misclassification rate. The 8 submitted variants are selected greedily: a new variant is kept only if it differs from all previous ones by at least 4 test-row predictions, ensuring genuine diversity in the submission set.

Enhanced ExtraTrees fallback (nb38)

Notebooks 36–38 produced a refined stand-alone fallback that matches the blend’s OOF error (0.0713):

- **Instance weights:** $w_i = \text{rarity}_i^{1.5} \cdot \text{sparsity}_i^{0.5}$, clipped to $[0.5, 20]$. Rarity is $1/f_i$ where f_i is the hash-frequency of recipe i ; sparsity is $(m - \text{nnz}_i)/m$ where m is the maximum nonzero count.
- **Multi-seed bagging:** 30 seeds $\times$ 500 trees averaged to reduce variance near the decision boundary.
- **Cosine-KNN blend:** 10% contribution from the 3 cosine-nearest training neighbours (fitted on ℓ_2 -normalised vectors), adding a local-similarity signal for near-duplicate test rows.

4 Ingredient Interpretation

ExtraTrees feature importances

Impurity-based importances from the full-data ExtraTrees model (700 trees, averaged over all splits):

Feature	Importance	Feature	Importance
tfidf_sum	0.0601	flour_present	0.0353
nonzero_mean	0.0476	water_present	0.0301
flour	0.0471	oil	0.0292
concentration	0.0432	salt	0.0277
water	0.0404	white.meat	0.0249
tfidf_max	0.0371	sugar	0.0244

The four top features (`tfidf_sum`, `nonzero_mean`, `concentration`, `tfidf_max`) are all row-level summaries of recipe richness and specificity. Their dominance reflects a structural pattern: **Italian recipes in this dataset tend to use fewer, more specific ingredients at higher TF-IDF weights**, while American recipes spread weight more evenly across a wider set of common ingredients.

Among raw ingredients, `flour` is the single strongest predictor. Its high TF-IDF weight in a recipe signals baked or pasta dishes (breads, focaccia, fresh pasta) that are strongly Italian. `water` and `salt` are heavily used in both cuisines but their precise TF-IDF patterns differ; the model exploits this through split thresholds rather than a simple presence/absence signal.

Logistic regression coefficients (L2, C = 1)

Coefficients from the L2 logistic model trained on 85 features; positive value $\Rightarrow$ evidence for Italian, negative $\Rightarrow$ evidence for American.

Strongest Italian indicators:

- `oil_present` (+0.85) — olive oil as a fat base
- `onion` TF-IDF (+0.75) — intense onion use in sauces
- `parsley` TF-IDF (+0.66) — Italian herb staple
- `honey` TF-IDF (+0.67) — Italian desserts / dressings
- `concentration` (+0.58) — fewer, more dominant flavours
- `wine` TF-IDF (+0.43) — wine-braised dishes
- `flour_present` (+0.39) — pasta and bread recipes

Strongest American indicators:

- | | |
|---|---|
| • nonzero_mean (−1.09) — many equally weighted ingredients | • white.meat (−0.94) — chicken-centric dishes |
| • garlic_present (−1.04) — garlic at low intensity | • water_present (−0.76) — water as a base ingredient |
| • onion_present (−0.98) — onion at low intensity | • fruits (−0.57) — fruit salads and desserts |
| | • honey_present (−0.59) — honey at low intensity |

Raw TF-IDF vs. binary presence: an important contrast

Several ingredients appear with opposite signs for their raw TF-IDF value and their binary presence indicator (e.g., **onion** +0.75 vs. **onion_present** −0.98). This is not a modelling artefact: it reflects a genuine culinary distinction. When onion is *present at all* (binary indicator active), the recipe tends to be American — onion is a common, low-intensity American ingredient. When onion appears with a *high TF-IDF weight*, however, it becomes a strong Italian signal, because Italian cooking uses a small set of ingredients in concentrated forms (soffritto, aglio-olio). The same logic applies to **honey**, **garlic**, and **oil**. This intensity-vs.-presence contrast is the key structural pattern separating the two cuisines in this TF-IDF feature space.

Sparsity tiers and class overlap

Recipes are naturally grouped by their number of non-zero ingredients:

- **Tier 0** (≤ 3 ingredients, ~52 fallback test rows): Very sparse; the small feature footprint makes classification inherently ambiguous. OOF error $\approx 8.5\%$.
- **Tier 1** (4–7 ingredients, ~373 fallback test rows): The largest bucket. OOF error $\approx 8.8\%$. The model is most prone to confusing American multi-ingredient dishes with Italian ones here.
- **Tier 2** (≥ 8 ingredients, ~379 fallback test rows): Dense recipes. OOF error $\approx 6.0\%$. When enough ingredients are present, the combination pattern becomes highly discriminative.

Italian recipes are somewhat over-represented in Tier 2 (more high-density recipes), while American recipes are slightly more common in the low-density tiers. This explains why **nonzero_mean** and **concentration** are among the most important features: they proxy the sparsity-tier membership without requiring an explicit tier assignment.